KPR INSTITUTE OF ENGINEERING AND TECHNOLOGY

Department of Artificial Intelligence and Data Science

ASSIGNMENT II

EXPLORATORY DATA ANALYSIS AND VISUALIZATION

Stock Price Analysis Using EDA and LSTM

Detail	Information
Course Code and Title	U21ADP05 — Exploratory Data Analysis and Visualization
Student Name	CHANDRU S
Roll Number	24AD020
Department	Artificial Intelligence and Data Science
Year / Semester	III Year / V Semester
Academic Year	2026–2027 (Odd Semester)
Submission Date	15 October 2026

Case study of historical stock-market prices, statistical patterns, visualization, and time-series forecasting using a Long Short-Term Memory network.

Academic project report. Forecasting illustrations in this document are explicitly marked as illustrative; replace them with outputs generated from the selected real dataset before submission.

Abstract

This report presents a workflow for exploratory data analysis (EDA), statistical visualization, and time-series forecasting of historical stock prices. The proposed dataset contains daily market fields such as Date, Open, High, Low, Close, and Volume, with derived variables including daily return and moving averages. The workflow inspects data types, missing values, duplicate dates, invalid prices, distributions, temporal patterns, and relationships among market variables. At least five visualizations are specified and interpreted, including a price trend, return distribution, trading-volume series, correlation heatmap, and return-volume scatterplot. An LSTM neural network is proposed to learn sequential patterns from scaled historical closing prices. The report describes chronological train, validation, and test partitions and evaluation using MAE, RMSE, and MAPE. Figures in this generated report use simulated illustrative values to demonstrate presentation and must not be interpreted as real market findings or trained-model results. The appendix provides reproducible Python code for running the analysis on a downloaded CSV.

Keywords: Exploratory Data Analysis, stock market, time series, LSTM, forecasting, data visualization.

Report Organization

• 1. Introduction and Objectives

• 2. Dataset Description

• 3. Data Understanding and Preprocessing

• 4. EDA and Visualization

• 5. LSTM Model Development

• 6. Evaluation and Interpretation

• 7. Conclusion and Future Scope

• 8. Literature Review

• 9. References

• Appendix A. Python Code

1. Introduction and Objectives

1.1 Introduction

Stock markets produce time-indexed records of prices and trading activity. Exploratory analysis can help describe historical behaviour, variability, unusual observations, and relationships between market fields. A closing-price series can be studied with moving averages, daily returns, and volume indicators. A forecasting model may learn patterns from past observations, but market prices are influenced by news, macroeconomic conditions, company events, and investor behaviour that are not necessarily represented in a price-only dataset.

1.2 Problem Statement

The project investigates how historical stock-price variables change over time and whether an LSTM sequence model can estimate a future closing price from a fixed window of past observations. The analysis must avoid random shuffling of time series, data leakage during scaling, and unsupported claims that a forecast guarantees future market performance.

1.3 Aim

To apply EDA, descriptive statistics, visualization, preprocessing, and an LSTM-based forecasting workflow to a real-world stock-price dataset.

1.4 Objectives

• Inspect dataset structure, date range, field types, and descriptive statistics.

• Handle missing values, duplicate dates, and invalid price/volume records.

• Create at least five visualizations and interpret their purpose and insights.

• Derive returns and moving averages for temporal exploration.

• Prepare chronological training, validation, and test sequences without leakage.

• Build and evaluate an LSTM model using MAE, RMSE, and MAPE.

• Discuss limitations, future improvements, and reproducibility.

2. Dataset Description

The recommended dataset is a historical daily stock-price CSV. The S&P; 500 historical dataset on Kaggle provides daily Date, Open, High, Low, Close, Volume, and ticker-name fields; a single ticker can be selected for a focused case study. The dataset source should be downloaded by the student and cited in the final submission. See the Kaggle dataset reference [1].

Field	Meaning	Role / Type
Date	Trading date	Datetime index
Open	Opening price	Numeric input
High	Highest price during session	Numeric input
Low	Lowest price during session	Numeric input
Close	Closing price	Forecast target
Volume	Shares traded	Numeric input
Name / Symbol	Ticker identifier, if present	Category / filter
Daily_Return	Percentage change in Close	Derived numeric
SMA_20 / SMA_50	20- and 50-session moving averages	Derived numeric

2.1 Dataset Selection and Scope

Choose one company ticker (for example, GOOGL or another ticker available in the selected source) and sort its rows by date in ascending order. Record the exact source URL, download date, ticker, date range, row count, and field count in the final version. The number of records varies according to ticker and date range; it must be measured from the downloaded CSV rather than assumed.

2.2 Practical Application

The workflow demonstrates how a data analyst can summarize historical behaviour and evaluate a forecasting model. It is intended for academic experimentation, not as investment advice or a trading recommendation.

3. Data Understanding and Preprocessing

3.1 Data Inspection

Use Pandas to load the CSV and inspect shape, column names, data types, head/tail records, descriptive statistics, missing-value counts, and duplicate dates. Convert Date to a datetime type and sort records chronologically. If the source contains multiple tickers, filter the selected symbol before building sequences.

3.2 Missing Values and Duplicates

Check missing values separately for each column. Remove exact duplicate rows and investigate repeated dates. Do not automatically fill every missing price: a missing trading record and a non-trading holiday are different cases. For a basic single-ticker analysis, rows missing the target Close value may be removed; other missing values should be handled using a documented rule.

3.3 Outlier and Validity Checks

Check that prices are positive and that High is not below Low. Investigate unusual returns and volume spikes with boxplots or quantiles. Extreme market movements may be real events, so do not delete them solely because they appear statistically unusual. Adjusted prices are preferable when the source provides them and the analysis needs to account for splits and dividends.

3.4 Feature Engineering

Daily return is calculated as Close.pct_change(). Moving averages are rolling means over the preceding 20 and 50 trading sessions. Rolling calculations create initial missing values; these rows should be handled deliberately. Derived variables support EDA, but the simple LSTM appendix uses scaled closing prices to keep the target and model workflow clear.

3.5 Chronological Split and Scaling

Time-series data must preserve chronological order. A recommended split is the earliest 70% of observations for training, the next 15% for validation, and the latest 15% for testing. Fit the scaler on the training segment only and apply that fitted scaler to validation and test data. This prevents future information from influencing training transformations.

Partition	Recommended share	Purpose
Training	70%	Fit model parameters and scaler
Validation	15%	Select settings and monitor generalization
Test	15%	Final evaluation once model choices are fixed

3.6 Sequence Construction

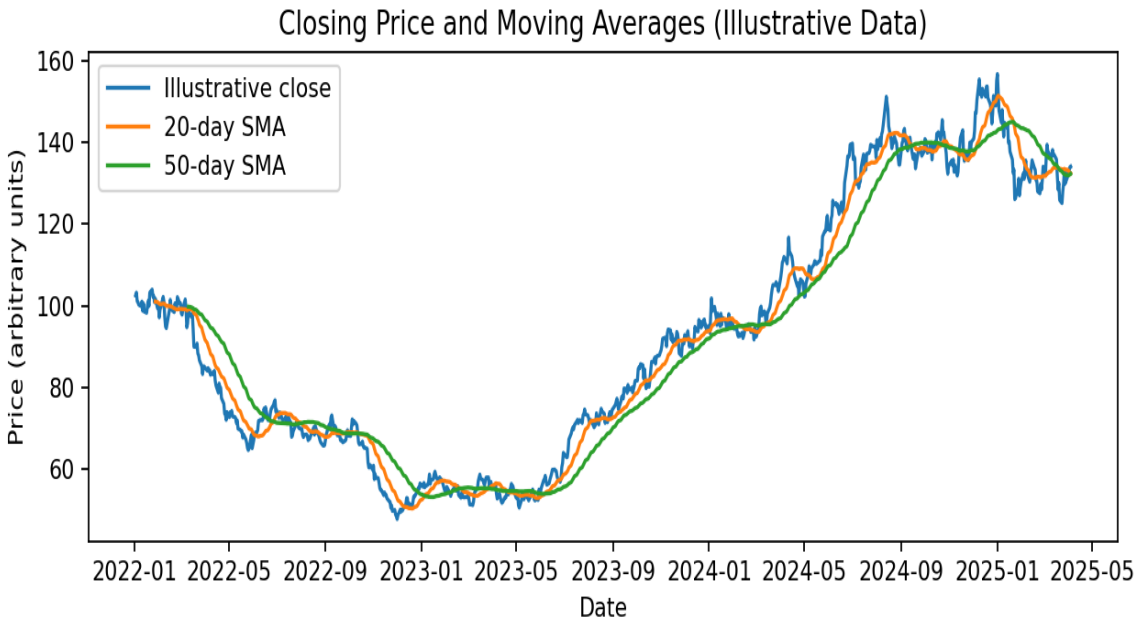
For a look-back window of 60 sessions, each model input contains the previous 60 closing prices and the target is the next closing price. Sliding windows are created in date order. Validation and test windows may use earlier historical observations as context, but their target dates must remain within the corresponding partition. No target values from the future may be included in a feature window.

3.7 Seven Stages of Visualization

Stage	Application
1. Acquire	Download the selected ticker CSV and record its source.
2. Parse	Load data and inspect schema and types.
3. Filter	Select ticker, sort dates, and address invalid rows.
4. Mine	Compute summary statistics, returns, and correlations.
5. Represent	Choose line plots, histograms, scatterplots, and heatmaps.
6. Refine	Add clear titles, units, labels, legends, and scales.
7. Interact	Optionally add hover tooltips or date-range filtering.

4. Exploratory Data Analysis and Visualization

4.1 Closing Price and Moving Averages



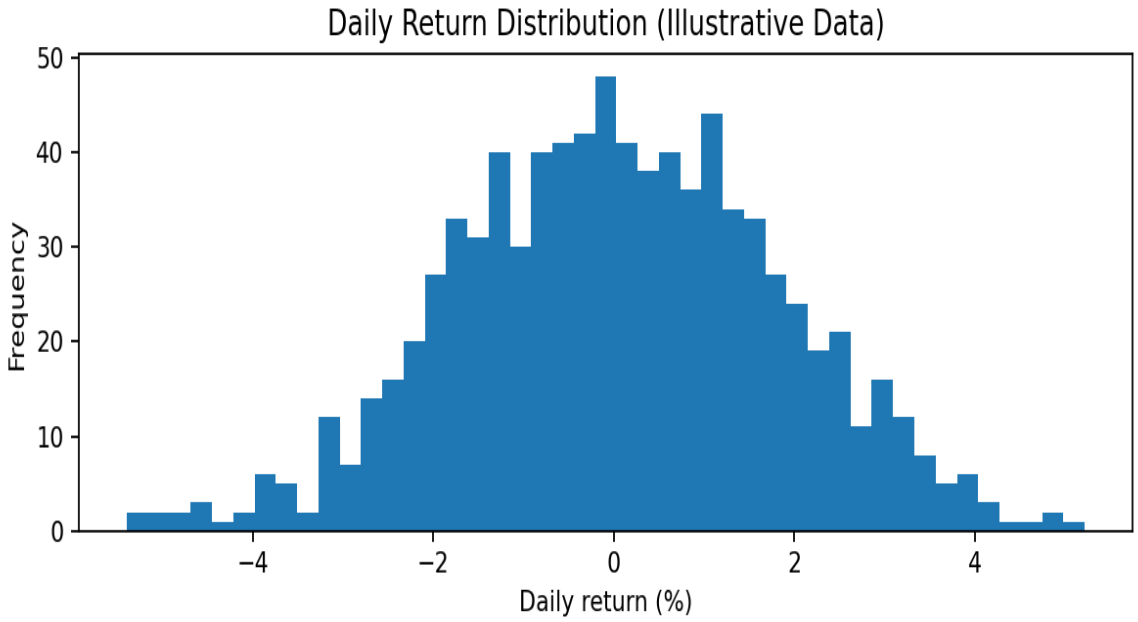
What it shows: A closing-price series and 20-/50-session moving averages. **Why created:** To examine long-run direction and smooth short-term fluctuations. **Interpretation:** Moving averages reduce day-to-day noise, but they lag the underlying price. The plotted values are simulated for layout demonstration; generate this chart from the downloaded dataset for actual findings.

4.2 Descriptive Statistical Summary

Calculate count, mean, median, standard deviation, minimum, quartiles, and maximum for Open, High, Low, Close, Volume, and Daily_Return. Report the computed values from the actual selected ticker. Price levels may trend over time, so summary statistics should be interpreted together with time-series plots rather than as a complete description of the market.

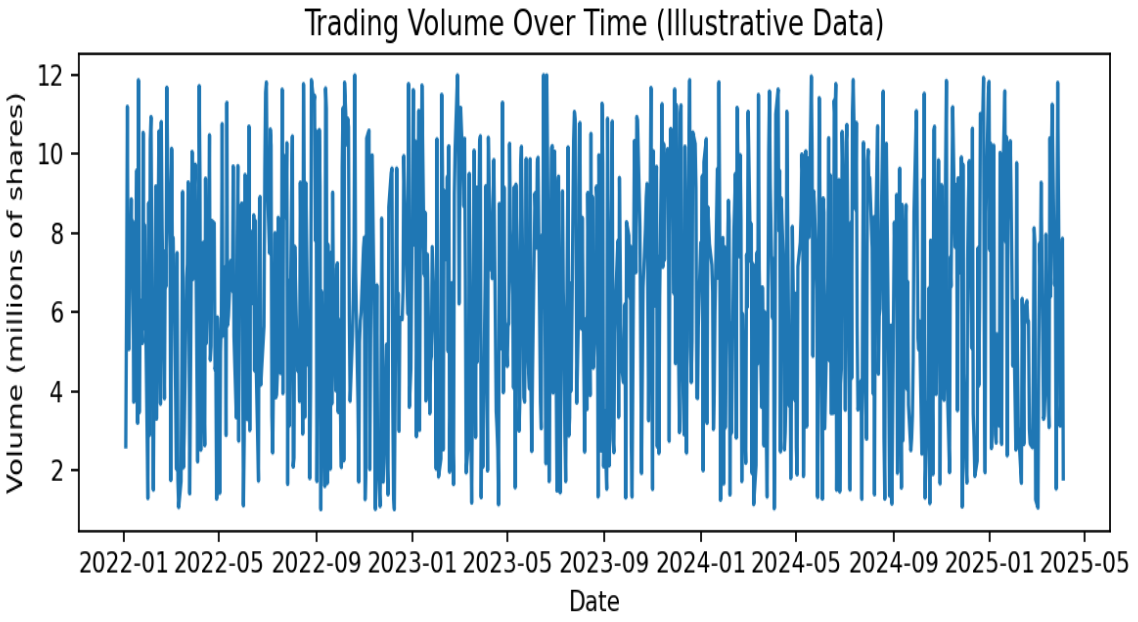
Statistic	Purpose
Mean / median	Typical level; comparison can indicate skew.
Standard deviation	Absolute spread of the observed values.
Quartiles / IQR	Middle 50% range and potential extreme observations.
Minimum / maximum	Observed endpoints; inspect extreme records.
Daily return mean / standard deviation	Average daily change and volatility proxy.

4.3 Daily Return Distribution



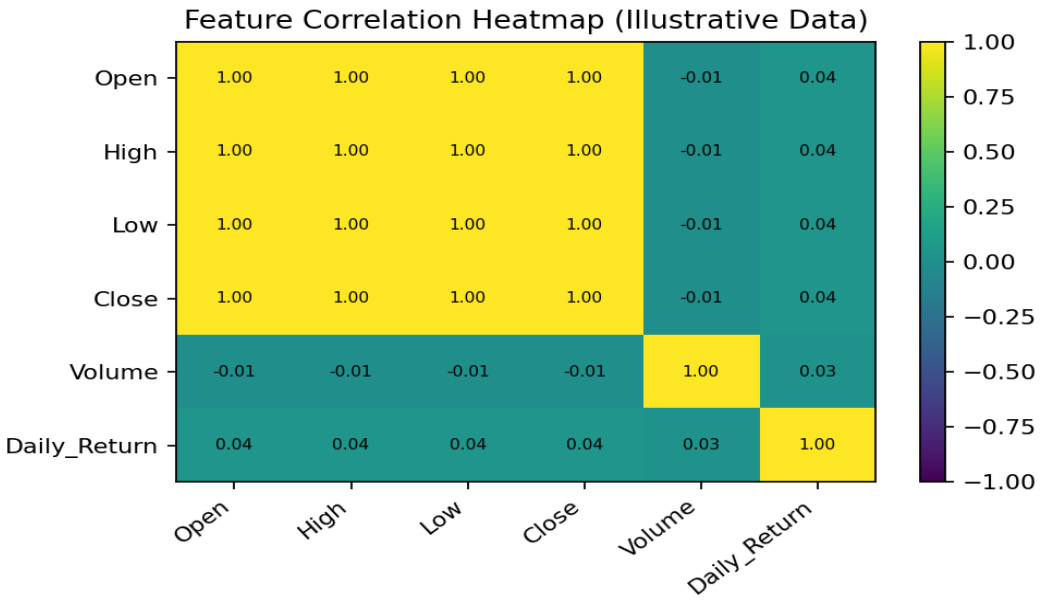
What it shows: Frequency distribution of percentage daily returns. **Why created:** To inspect typical daily changes, asymmetry, and extreme movements. **Interpretation:** Returns often cluster near zero with occasional larger movements, but the actual shape depends on the chosen ticker and period. Do not infer normality without additional tests. Figure uses simulated illustrative data.

4.4 Trading Volume Over Time



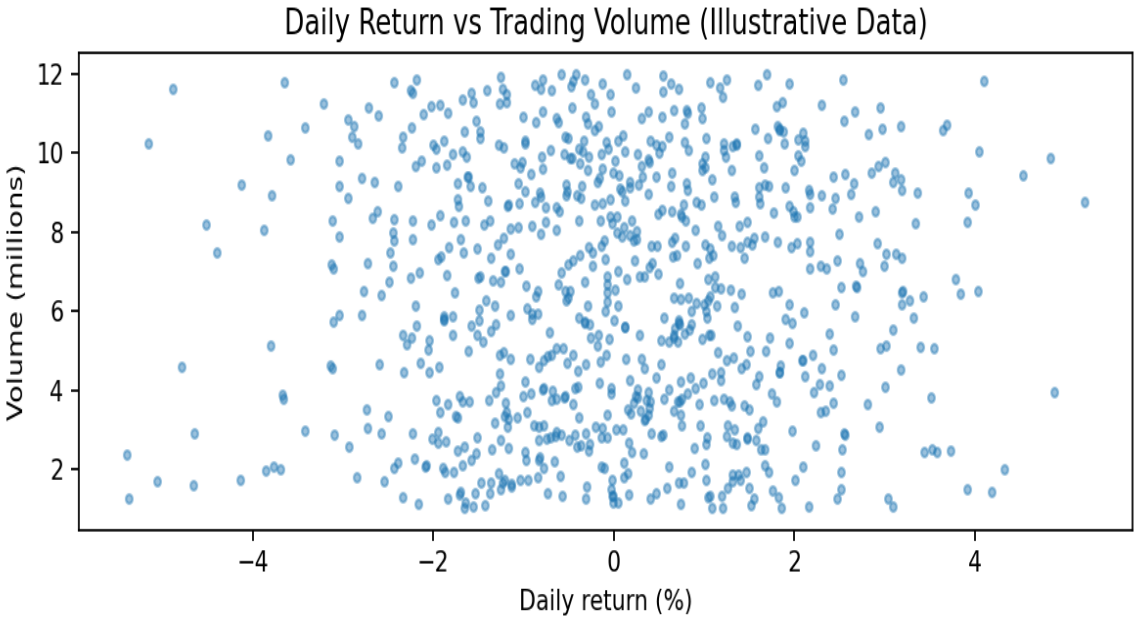
What it shows: Daily share volume across the date range. **Why created:** To identify periods of unusually high or low trading activity. **Interpretation:** Volume spikes may coincide with earnings, announcements, or market-wide events; this chart alone cannot establish the reason. Figure uses simulated illustrative data.

4.5 Correlation Heatmap



What it shows: Pairwise linear correlations among numerical market fields and derived returns. **Why created:** To identify redundant price features and possible relationships for further investigation. **Interpretation:** Open, High, Low, and Close can be strongly correlated because they measure the same session's price level. Correlation does not prove causation; the displayed coefficients are illustrative.

4.6 Return Versus Volume



What it shows: The relationship between daily percentage returns and trading volume. **Why created:** To inspect whether large price moves appear alongside unusually high trading activity. **Interpretation:** A visible pattern, if present in real data, should be quantified and checked for outliers. Volume is often highly skewed, so a log scale may be useful. Figure uses simulated data.

4.7 Additional Visualization: High–Low Range

A useful sixth visualization is a histogram or boxplot of the daily range, defined as High – Low, or a time-series line for the percentage range (High – Low) / Close. This helps compare intraday price dispersion across sessions. Use clear units and inspect whether large ranges cluster around particular periods.

4.8 Key EDA Questions

• Does the closing price show a long-term upward or downward trend during the selected period?

• Are daily returns centred near zero, skewed, or heavy-tailed?

• Are there missing records, duplicate dates, or impossible OHLC values?

• Do volume spikes occur with unusually large absolute returns?

• How strongly are Open, High, Low, and Close correlated?

• Does volatility vary over time, suggesting periods of clustered movement?

4.9 Statistical Interpretation

Daily returns are preferable to raw price levels for comparing relative changes across different price ranges. Standard deviation of daily returns is a simple volatility measure, but it does not capture every aspect of risk. Rolling standard deviation can show whether volatility changes over time. Any numerical conclusions in the final submission must be calculated from the selected real CSV; this report deliberately avoids presenting simulated values as real stock-market statistics.

5. LSTM Model Development

5.1 Why LSTM?

Long Short-Term Memory (LSTM) is a recurrent neural network architecture designed to learn sequential dependencies. Its input, forget, and output gates regulate information flow through a cell state. This can help model temporal patterns, although it does not guarantee better performance than simple baselines.

5.2 Proposed Architecture

Layer	Configuration	Purpose
Input	60 prior Close observations	Sequence context
LSTM	50 units	Learn temporal representations
Dropout	0.2	Regularization
Dense	1 neuron	Predict next scaled Close
Loss	Mean Squared Error	Penalize squared prediction error
Optimizer	Adam	Update trainable weights

5.3 Training Parameters

Parameter	Suggested value
Look-back window	60 trading sessions
Batch size	32
Maximum epochs	50
Validation	Chronological validation set
Early stopping	Monitor validation loss; patience 5
Random seed	Set and report for reproducibility

5.4 Baseline Comparison

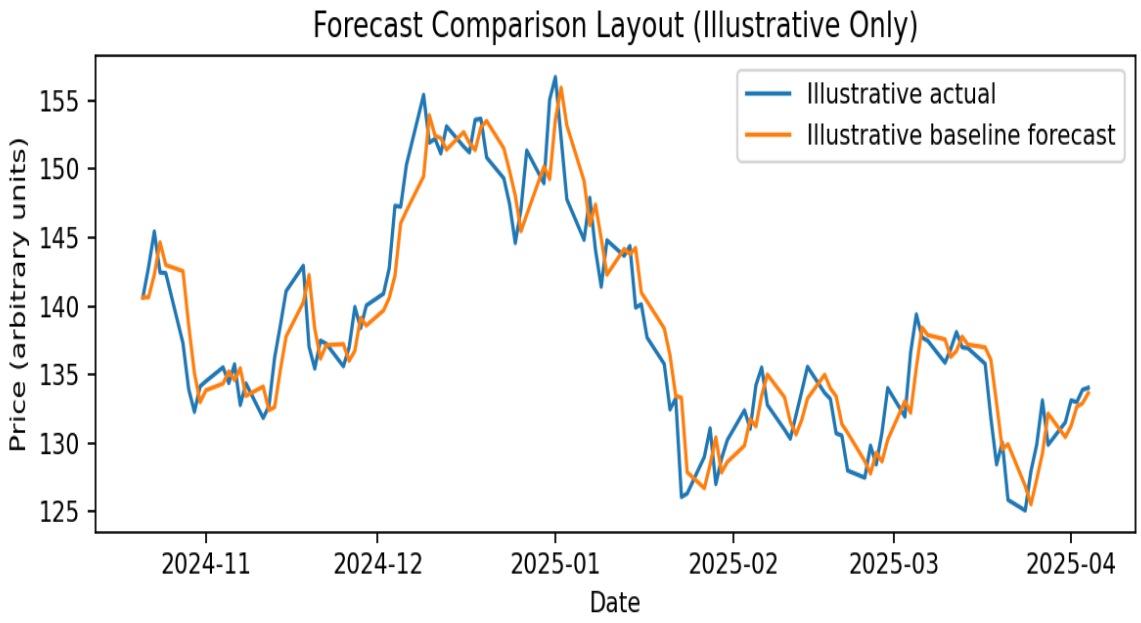
Compare LSTM forecasts with a naïve baseline that predicts the next closing price as the latest observed closing price. This baseline is important because stock prices often have strong short-term persistence. A complex model should only be considered useful if it improves on a baseline on untouched test dates.

6. Model Evaluation and Result Visualization

6.1 Evaluation Metrics

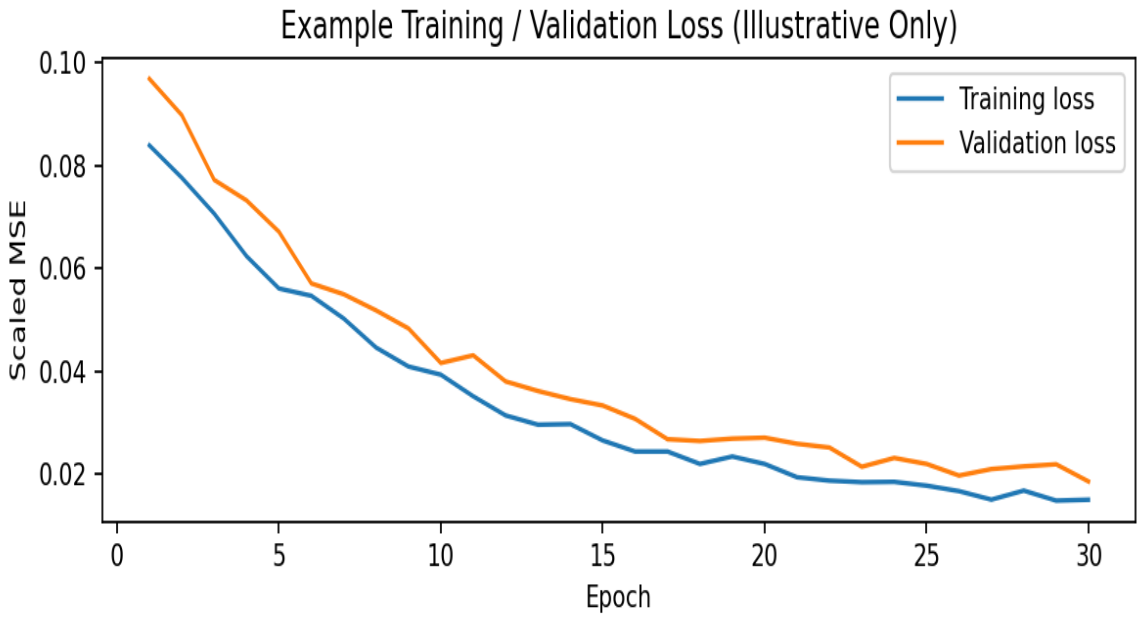
Metric	Definition / interpretation
MAE	Mean absolute error in the original price units; lower is better.
RMSE	Square root of mean squared error; penalizes large errors more.
MAPE	Mean absolute percentage error; can be unstable when actual values approach zero.
Naïve baseline	Compare against the previous observed closing price.

6.2 Forecast Plot



This figure demonstrates the expected layout for plotting predictions against actual closing prices. It is not a trained LSTM result and must be replaced by the output of the appendix code after training on the real dataset.

6.3 Training and Validation Loss



Training and validation loss curves help diagnose learning and overfitting. A widening gap may indicate overfitting; both curves remaining high may indicate underfitting. The plotted curve is illustrative only, not evidence of an actual model run.

6.4 Results Table and Interpretation

After running the code, fill in the following table using the actual test partition. Do not copy illustrative numbers from sample plots. Inverse-transform predictions before calculating price-unit errors.

Evaluation item	Actual result to record
Ticker and test date range	Fill from selected CSV
Number of test predictions	Print from test array
LSTM MAE	Compute from y_test and prediction
LSTM RMSE	Compute from y_test and prediction
LSTM MAPE	Compute only with a documented zero-value rule
Naïve baseline MAE / RMSE	Compute on identical test dates
LSTM vs baseline	State whether test errors improve

6.5 Error Analysis

Inspect residuals (actual minus predicted values) over time and plot their distribution. Large errors may occur around abrupt price movements that a model trained on past prices cannot anticipate. Evaluate performance across the entire test period and, if possible, use rolling-origin validation to check whether the result is stable across different market regimes.

6.6 Limitations

- A price-only model cannot anticipate all news or macroeconomic events.
- A single historical period may not represent future market conditions.
- Scaling or splitting incorrectly can leak future information into training.
- MAPE can be misleading when actual values are zero or near zero.
- A lower test error does not guarantee profitability after fees, slippage, or risk.

7. Conclusion and Future Scope

7.1 Conclusion

This report defines a complete EDA and time-series modelling workflow for stock-price analysis. The proposed analysis inspects price and volume fields, cleans and orders the data, derives daily returns and moving averages, and uses complementary visualizations to study trend, distribution, correlation, and trading activity. An LSTM model is proposed to forecast the next closing price from a rolling historical window, with chronological validation and test partitions. Model quality should be judged using MAE and RMSE and compared against a naïve baseline. Since the charts included in this generated document use simulated data, no real-stock conclusion or model-performance claim is made here. The final submission should include the selected ticker, actual dataset statistics, and outputs generated by running the appendix code.

7.2 Future Scope

• Use adjusted closing prices where available to account for splits and dividends.

• Compare LSTM with naïve, ARIMA, and tree-based models.

• Add technical indicators and volume features, ensuring all inputs are available at prediction time.

• Use rolling-origin cross-validation and test on multiple market periods.

• Explore probabilistic forecasts and prediction intervals rather than a single point estimate.

• Document transaction costs and risk before any trading-related evaluation.

7.3 Practical Relevance

The project demonstrates reproducible data preparation, visualization design, sequential modelling, and careful evaluation. It is a learning exercise in data science and should not be interpreted as financial advice.

8. Literature Review

Fry [2] presents a structured visualization workflow spanning data acquisition, parsing, filtering, mining, representation, refinement, and interaction. This framework is reflected in the report by progressing from dataset selection and cleaning to statistical summaries and labelled visualizations.

Hochreiter and Schmidhuber [3] introduced the Long Short-Term Memory architecture to address learning challenges in recurrent networks. LSTM cells use gated information flow and a cell state to learn dependencies across sequences. Their use in financial time-series prediction is an application of the architecture rather than a guarantee of forecasting accuracy.

The scikit-learn documentation [4] describes time-series-aware validation approaches and emphasizes that temporal observations should not be randomly mixed when the goal is to predict future values. The TensorFlow/Keras documentation [5] provides the LSTM layer and training interfaces used in the appendix.

The Kaggle S&P; 500 historical dataset [1] provides a practical source of daily OHLCV observations for exploratory work. Dataset documentation should be reviewed to understand fields, source limitations, and date coverage. The selected ticker and downloaded version should be recorded so the analysis can be reproduced.

9. References

[1] C. Nugent, “S&P; 500 Stock Data,” Kaggle. Available: https://www.kaggle.com/camnugent/sandp500/data

[2] B. Fry, *Visualizing Data: Exploring and Explaining Data with the Processing Environment*. O’Reilly Media, 2008.

[3] S. Hochreiter and J. Schmidhuber, “Long Short-Term Memory,” *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.

[4] scikit-learn developers, “TimeSeriesSplit,” scikit-learn documentation. Available: https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.TimeSeriesSplit.html

[5] TensorFlow, “tf.keras.layers.LSTM,” TensorFlow API documentation. Available: https://www.tensorflow.org/api_docs/python/tf/keras/layers/LSTM

Appendix A. Python Code and Reproducibility

Install required libraries: pandas, numpy, matplotlib, scikit-learn, and tensorflow. Download a historical stock CSV, save it as stock_data.csv, and ensure it includes Date and Close columns. The code below creates chronological windows, fits scaling on training data only, trains an LSTM, plots predictions, and reports test errors.

```
# Stock Price EDA and LSTM: reproducible starter code
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.preprocessing import MinMaxScaler
from sklearn.metrics import mean_absolute_error, mean_squared_error
from tensorflow.keras import Sequential
from tensorflow.keras.layers import LSTM, Dense, Dropout
from tensorflow.keras.callbacks import EarlyStopping

# 1. Load and clean
df = pd.read_csv("stock_data.csv")
df["Date"] = pd.to_datetime(df["Date"], errors="coerce")
df["Close"] = pd.to_numeric(df["Close"], errors="coerce")
df = (df.dropna(subset=["Date", "Close"])
        .drop_duplicates()
        .sort_values("Date")
        .drop_duplicates(subset=["Date"], keep="last"))
df = df[df["Close"] > 0].reset_index(drop=True)

# 2. EDA and engineered features
df["Daily_Return"] = df["Close"].pct_change()
df["SMA_20"] = df["Close"].rolling(20).mean()
df["SMA_50"] = df["Close"].rolling(50).mean()
print("Shape:", df.shape)
print(df.describe(include="all"))
print("Missing values:\\n", df.isna().sum())

plt.figure(figsize=(10, 4))
plt.plot(df["Date"], df["Close"], label="Close")
plt.plot(df["Date"], df["SMA_20"], label="SMA 20")
plt.plot(df["Date"], df["SMA_50"], label="SMA 50")
plt.legend(); plt.title("Closing price and moving averages"); plt.tight_layout()
plt.show()

plt.figure(figsize=(7, 4))
df["Daily_Return"].dropna().hist(bins=40)
plt.title("Daily return distribution"); plt.tight_layout(); plt.show()

# 3. Chronological train/validation/test split
values = df["Close"].to_numpy().reshape(-1, 1)
n = len(values)
train_end, val_end = int(n * .70), int(n * .85)
scaler = MinMaxScaler()
scaler.fit(values[:train_end])                # fit ONLY on training segment
scaled = scaler.transform(values)
lookback = 60

def make_sequences(series, start_target, end_target, lookback):
    X, y, target_idx = [], [], []
    for i in range(max(start_target, lookback), end_target):
        X.append(series[i-lookback:i, 0])
        y.append(series[i, 0])
        target_idx.append(i)
    return np.array(X), np.array(y), np.array(target_idx)

X_train, y_train, _ = make_sequences(scaled, lookback, train_end, lookback)
X_val, y_val, _ = make_sequences(scaled, train_end, val_end, lookback)
X_test, y_test, test_idx = make_sequences(scaled, val_end, n, lookback)
X_train = X_train.reshape((-1, lookback, 1))
X_val = X_val.reshape((-1, lookback, 1))
X_test = X_test.reshape((-1, lookback, 1))

# 4. LSTM model
model = Sequential([
    LSTM(50, input_shape=(lookback, 1)),
    Dropout(0.2),
    Dense(1)
])
model.compile(optimizer="adam", loss="mse")
stop = EarlyStopping(monitor="val_loss", patience=5,
                     restore_best_weights=True)
history = model.fit(X_train, y_train, validation_data=(X_val, y_val),
                    epochs=50, batch_size=32, callbacks=[stop],
                    shuffle=False, verbose=1)

# 5. Evaluate in original price units
pred_scaled = model.predict(X_test, verbose=0)
y_true = scaler.inverse_transform(y_test.reshape(-1, 1)).ravel()
y_pred = scaler.inverse_transform(pred_scaled).ravel()
mae = mean_absolute_error(y_true, y_pred)
rmse = np.sqrt(mean_squared_error(y_true, y_pred))
mape = np.mean(np.abs((y_true - y_pred) / y_true)) * 100
naive = df["Close"].to_numpy()[test_idx - 1]
print(f"Test MAE={mae:.4f}, RMSE={rmse:.4f}, MAPE={mape:.2f}%")
print("Naive baseline MAE:",
      mean_absolute_error(y_true, naive))
print("Naive baseline RMSE:",
      np.sqrt(mean_squared_error(y_true, naive)))

# 6. Result plots
plt.figure(figsize=(10, 4))
```

```
plt.plot(df["Date"].iloc[test_idx], y_true, label="Actual")
plt.plot(df["Date"].iloc[test_idx], y_pred, label="LSTM")
plt.legend(); plt.title("Test set: actual vs LSTM forecast")
plt.tight_layout(); plt.show()

plt.figure(figsize=(7, 4))
plt.plot(history.history["loss"], label="Training loss")
plt.plot(history.history["val_loss"], label="Validation loss")
plt.legend(); plt.title("Training and validation loss")
plt.tight_layout(); plt.show()

# Note: inspect source columns and date coverage before reporting results.
# Predictions are educational estimates, not investment advice.
```

Submission checklist: add the actual ticker and date range; replace illustrative figures with generated charts; paste real metrics; include the dataset link and notebook/script in your public GitHub repository; verify the code runs from a clean environment.